



DEPARTMENT OF
COMPUTER SCIENCE



SOFTWARE DESIGN CONCEPTS

JEREMIAH O. BANDELE

PhD Electrical/Electronic Engineering
University of Nottingham



LEARNING OBJECTIVES

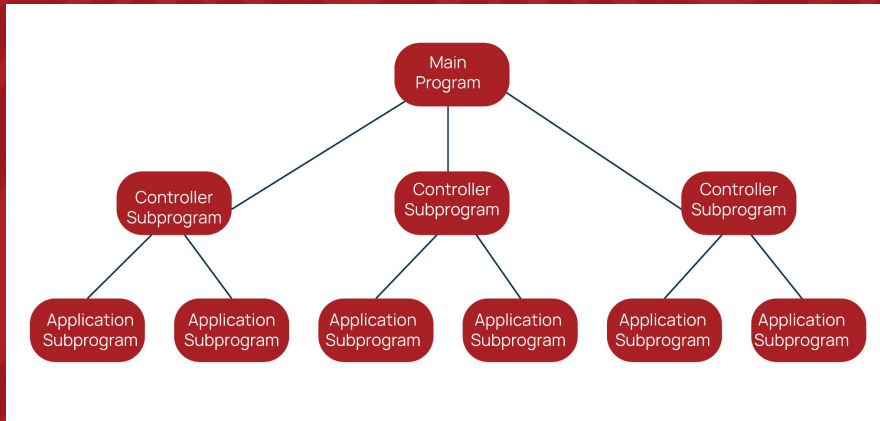
At the end of the lesson, you should be able to:

- Describe software architecture and why we need software architecture to solve problems.
- Outline various types of software patterns.
- Discuss different forms of software design approaches.



SOFTWARE ARCHITECTURE

Software architecture refers to the structure of the system, which contains various components of a program/system, the attributes (properties) of those components and the relationship between them. The software architecture enables the software engineers to analyze the software design efficiently.





SOFTWARE ARCHITECTURE: WHY

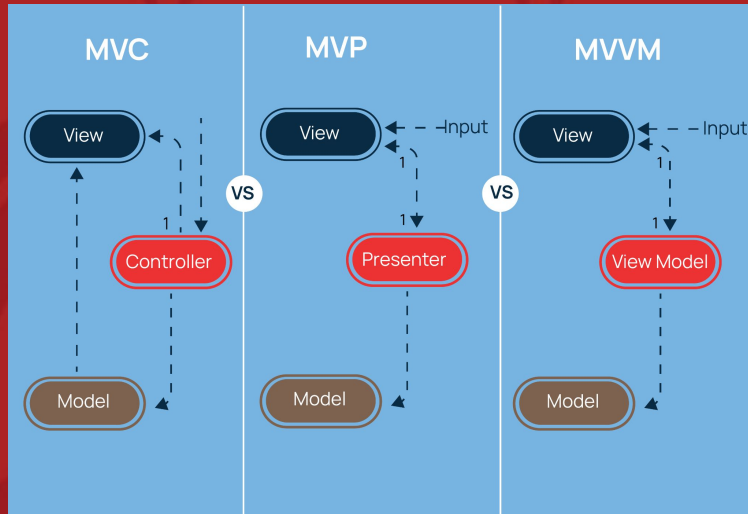
A software architecture:

- Enables efficient communication between all the interested stakeholders.
- Highlights early design decisions that affects activities such as coding and testing.
- Creates intellectual models of how the system is organized into components and how these components interact with each other.



SOFTWARE PATTERN

This provides a description of the solution to a recurring design problem of some specific domain in such a way that the solution can be used again and again.





SOFTWARE PATTERN

The objective of each pattern is to provide an insight to a designer who can determine the following:

- Whether the pattern can be reused.
- Whether the pattern is applicable to the current project.
- Whether the pattern can be used to develop a similar but functionally or structurally different design pattern.



SOFTWARE PATTERN: ARCHITECTURAL

High-level strategies that refer to the overall structure and organization of a software system.

They define the elements of a software system such as subsystems, components, classes, etc.

They also indicate the relationship between the elements along with the rules and guidelines for specifying these relationships.

Note that architectural patterns are often considered equivalent to software architecture.



SOFTWARE PATTERN: DESIGN

Medium-level strategies that are used to solve design problems.

They provide a means for the refinement of the elements (as defined by architectural pattern) of a software system or the relationship among them.

Specific design elements such as relationship among components or mechanisms that affect component-to-component interaction are addressed by design patterns.

Note that design patterns are often considered equivalent to software components.



SOFTWARE PATTERN: IDIOM

Low-level patterns, which are programming-language specific.

They describe the implementation of a software component, the method used for interaction among software components, etc., in a specific programming language.

Note that idioms are often termed as coding patterns.



SOFTWARE DESIGN APPROACHES

There are two generic approaches to software design:

- Top-down design
- Bottom-up design

Top-down and bottom-up approaches are not individually practicable. Instead, a good combination of both is used.

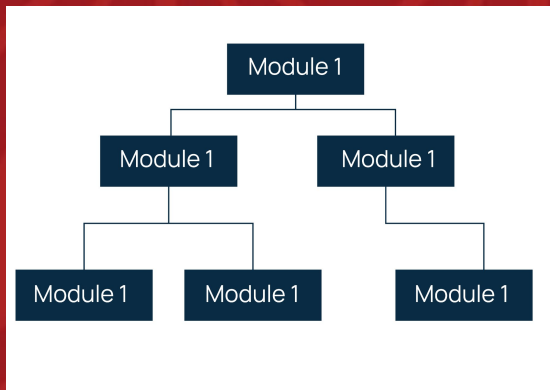


SOFTWARE DESIGN APPROACHES: TOP-DOWN

Top-down design takes the whole software system as one entity and then decomposes it to achieve more than one sub-system or component based on some characteristics.

Each sub-system or component is then treated as a system and decomposed further.

This process keeps on running until the lowest level of system in the top-down hierarchy is achieved.





SOFTWARE DESIGN APPROACHES: TOP-DOWN

Top-down design starts with a generalized model of system and keeps on defining the more specific part of it.

When all components are composed the whole system comes into existence.

Top-down design is more suitable when the software solution needs to be designed from scratch and specific details are unknown.

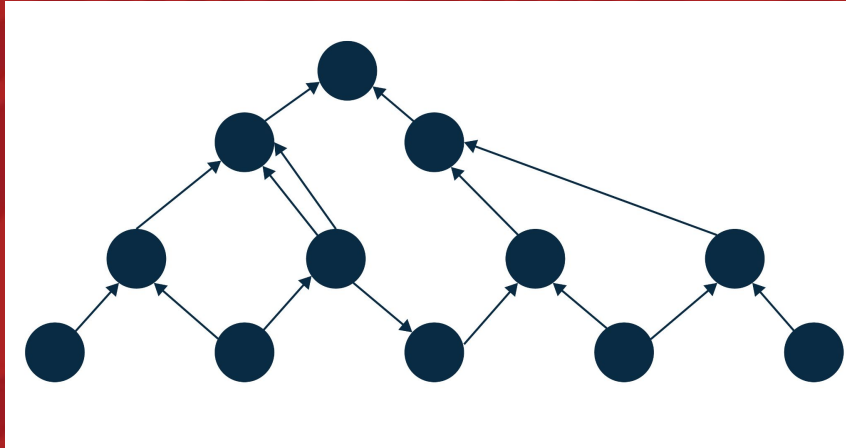


SOFTWARE DESIGN APPROACHES: BOTTOM-UP

The bottom up design model starts with most specific and basic components.

It proceeds with composing higher level of components by using basic or lower level components.

It keeps creating higher level components until the desired system is evolved as one single component.





SOFTWARE DESIGN APPROACHES: BOTTOM-UP

With each higher level, the amount of abstraction is increased.

Bottom-up strategy is more suitable when a system needs to be created from some existing system, where the basic primitives can be used in the newer system.



SUMMARY

- Software architecture refers to the structure of the system, which contains various components of a program/system.
- There are two generic approaches to software design: Top-down design, Bottom-up design



FURTHER READING RESOURCES

- Jalote, P. (1997). An Integrated Approach to Software Engineering. Germany: Springer.
- Buschmann, F. (1996). Pattern-Oriented Software Architecture, A System of Patterns. Singapore: Wiley.



DEPARTMENT OF
COMPUTER SCIENCE



THANK YOU